

NAME: Gaurika Mittal

BATCH: B15

SAP ID: 590034892

DATE: 27|08|2026

EXPERIMENT-3

1. Fundamentals of Errors and Debugging

Theory

Programming is not always error-free. While writing a program, programmers may make mistakes that prevent the program from working correctly. These mistakes are called errors. Some errors stop the program from compiling, while others allow the program to run but produce incorrect results. To identify and fix these problems, programmers use a process called debugging. Debugging helps in finding the exact cause of an error and correcting it so that the program produces the expected output. It is an important part of software development because it improves the quality and reliability of a program.

2. Classifications of Errors in C

1. Syntax Errors

- Occur when the rules or grammar of the C language are not followed.
- These errors are detected by the compiler before the program runs.
- The program cannot compile until the syntax errors are corrected.
- Common causes include missing semicolons, unmatched braces, and incorrect keywords.

2. Compilation (Type) Errors

- Occur when incompatible data types are used or variables/functions are not declared properly.
- These errors are identified during the compilation process.
- The program will not compile successfully until the errors are fixed.
- Common examples include using undeclared variables or assigning incompatible data types.

3. Linker Errors

- Occur after successful compilation when the linker cannot create the executable file.
- They usually happen because a function is declared but not defined or a required library is missing.
- The program cannot be executed until the missing references are resolved.
- An example is the "**undefined reference to 'main'**" error.

4. Runtime Errors

- Occur while the program is running.
- These errors are not detected by the compiler.
- They may cause the program to crash or terminate unexpectedly.
- Common examples include division by zero and accessing invalid memory locations.

5. Logical Errors

- Occur when the program compiles and runs successfully but produces incorrect results.
- These errors are caused by mistakes in the program logic or calculations.
- They are difficult to identify because the compiler does not report them.
- They are usually found by checking the output carefully or by using debugging techniques.

Why Does a Program Compile Successfully but Produce Incorrect Output?

A program may compile successfully but still produce incorrect output because the compiler only checks the syntax and data types, not the programmer's logic. If the code follows the rules of the C language, it will compile even if the calculations or conditions are incorrect. Therefore, logical errors can only be found by testing and debugging the program carefully.

Incorrect Code 1

```
#include <stdio.h>
```

```
int main()
```

```
{
```

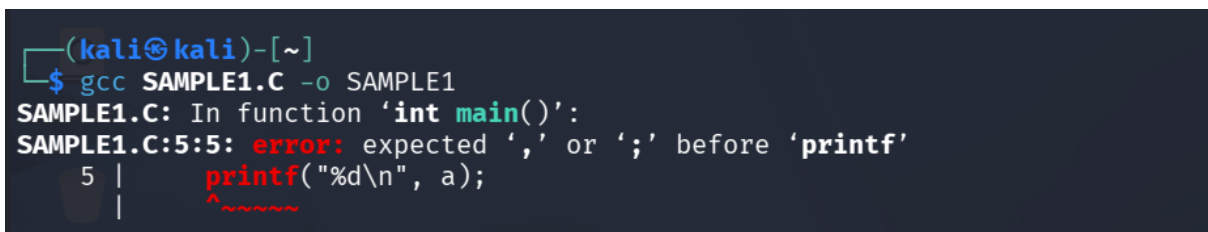
```
    int a = 10
```

```
    printf("%d\n", a);
```

```
    return 0;
```

```
}
```

Output



```
(kali㉿kali)-[~]  
$ gcc SAMPLE1.C -o SAMPLE1  
SAMPLE1.C: In function 'int main()':  
SAMPLE1.C:5:5: error: expected ',' or ';' before 'printf'  
  5 | printf("%d\n", a);  
    | ^~~~~~
```

Error Message

error: expected ',' or ';' before 'printf'

Error Type

Syntax Error

Explanation

The semicolon (;) is missing after `int a = 10`. Because of this, the compiler treats the next line as part of the previous statement and reports a syntax error.

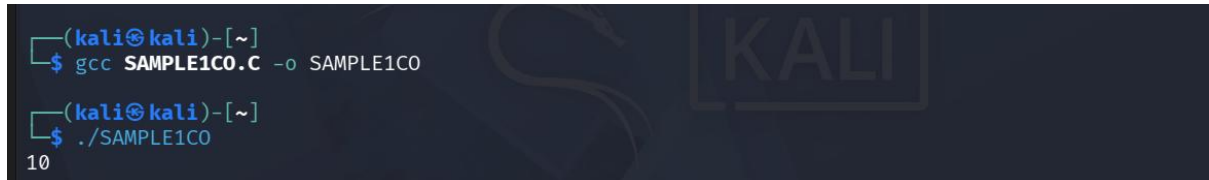
Correct Code

```
#include <stdio.h>
int main()
{
    int a = 10;
    printf("%d\n", a);
    return 0;
}
```

Correct Output

10

Screenshot of Correct Output



Conclusion

Adding the missing semicolon fixes the syntax error and allows the program to compile successfully.

3. Compiler Warnings (-Wall, -Wextra)

Theory

Compiler warnings are messages generated by the compiler to indicate possible problems in the program. Although warnings do not stop the program from compiling, they should never be ignored because they may lead to unexpected behaviour later. Using compiler options like -Wall and -Wextra helps programmers detect issues such as unused variables, suspicious code, and possible logical mistakes. Fixing compiler warnings makes the program more reliable and easier to maintain.

- -Wall: Enables a broad set of commonly useful compiler warnings.

- -Wextra: Enables additional warnings not covered by -Wall.

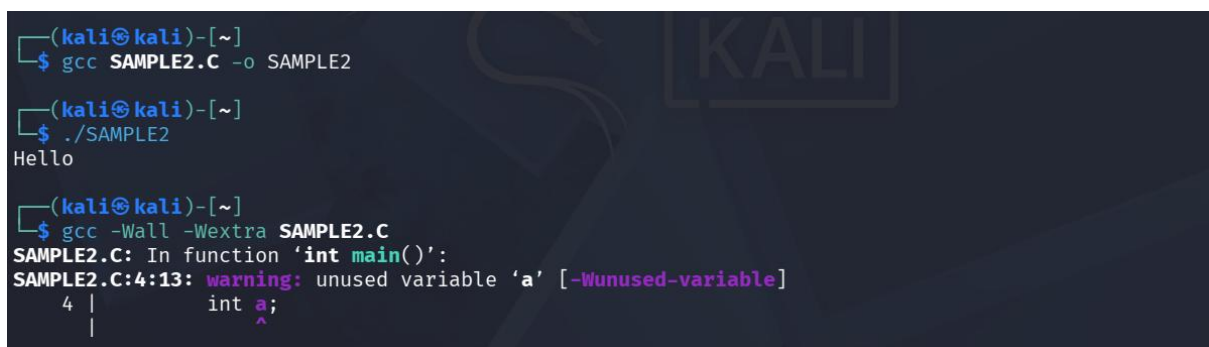
Incorrect Code 2

```
#include <stdio.h>
int main()
{
    int a;

    printf("Hello\n");

    return 0;
}
```

Screenshot of Wrong Code / Error Output



```
(kali㉿kali)-[~]
$ gcc SAMPLE2.C -o SAMPLE2

(kali㉿kali)-[~]
$ ./SAMPLE2
Hello

(kali㉿kali)-[~]
$ gcc -Wall -Wextra SAMPLE2.C
SAMPLE2.C: In function 'int main()':
SAMPLE2.C:4:13: warning: unused variable 'a' [-Wunused-variable]
    4 |         int a;
      |         ^
```

Error Type

Unused variable warning

Error Message / Output

warning: unused variable 'a'

Explanation

Variable 'a' is declared but never used. Remove it or use it.

Correct Code

```
#include <stdio.h>

int main(){

    printf("Hello\n");

    return 0;
}
```

Correct Output

Hello

Screenshot of Correct Output

```
(kali㉿kali)-[~]  
$ gcc -Wall -Wextra SAMPLE2.C  
  
(kali㉿kali)-[~]  
$ ./SAMPLE2  
Hello
```

Conclusion

After correcting the code, the program works as expected.

4. printf()-Based Debugging

Theory

One of the simplest methods of debugging is using **printf()** statements. By printing the values of variables and checking the program flow, a programmer can understand how the program is executing. This method is very useful for finding logical errors because it allows the programmer to observe changes in variables during each step of execution. Before using advanced debugging tools, many programmers first use `printf()` statements to identify the source of a problem.

Incorrect Code 3

```
#include <stdio.h>  
  
int main(void)  
{  
    int i, sum = 0;  
    for(i = 0; i < 5; i++)  
    {  
        printf("DEBUG: entering iteration i = %d\n", i);  
        sum = sum + i;  
        printf("DEBUG: sum updated to = %d\n", sum);  
    }  
    printf("Final Sum = %d\n", sum);  
    return 0;  
}
```

Screenshot of Wrong Code / Error Output

```
(kali㉿kali)-[~]  
$ gcc -Wall -Wextra SAMPLE3.C -o SAMPLE3  
  
(kali㉿kali)-[~]  
$ ./SAMPLE3  
DEBUG: entering iteration i = 0  
DEBUG: sum updated to = 0  
DEBUG: entering iteration i = 1  
DEBUG: sum updated to = 1  
DEBUG: entering iteration i = 2  
DEBUG: sum updated to = 3  
DEBUG: entering iteration i = 3  
DEBUG: sum updated to = 6  
DEBUG: entering iteration i = 4  
DEBUG: sum updated to = 10  
Final Sum = 10
```

Error Type

Loop logic bug

Error Message / Output

Program gives incorrect sum.

Correct Code

```
#include <stdio.h>  
  
int main(void)  
{  
    int i, sum = 0;  
    for(i = 1; i <= 5; i++)  
    {  
        printf("DEBUG: entering iteration i = %d\n", i);  
        sum = sum + i;  
        printf("DEBUG: sum updated to = %d\n", sum);  
    }  
    printf("Final Sum = %d\n", sum);  
    return 0;  
}
```

Correct Output

Final Sum = 15

Screenshot of Correct Output

```
(kali㉿kali)-[~]  
$ gcc -Wall -Wextra SAMPLE3.C -o SAMPLE3  
  
(kali㉿kali)-[~]  
$ ./SAMPLE3  
DEBUG: entering iteration i = 1  
DEBUG: sum updated to = 1  
DEBUG: entering iteration i = 2  
DEBUG: sum updated to = 3  
DEBUG: entering iteration i = 3  
DEBUG: sum updated to = 6  
DEBUG: entering iteration i = 4  
DEBUG: sum updated to = 10  
DEBUG: entering iteration i = 5  
DEBUG: sum updated to = 15  
Final Sum = 15
```

Conclusion

After correcting the code, the program works as expected.

5. Introduction to GDB (GNU Debugger)

Theory

GDB (GNU Debugger) is a powerful debugging tool used for C programs. It allows programmers to execute a program step by step, inspect variables, set breakpoints, and identify the exact location where an error occurs. GDB is especially useful for finding logical errors, runtime errors, and crashes. It helps programmers understand how the program behaves during execution and makes debugging more efficient.

Installation & Compilation for GDB

Update package list and install GDB on Linux systems

```
$ sudo apt update
```

```
$ sudo apt install gdb
```

Compile source code with debugging symbols enabled (-g)

```
$ cc -g -Wall -Wextra program.c -o program
```

Launch GDB with the compiled binary

```
$ gdb ./program
```

6. Common GDB Commands

- **break (b):** Used to set a breakpoint at a specific line or function. The program pauses at that point so you can inspect it.
- **run (r):** Starts the execution of the program from the beginning.
- **next (n):** Executes the next line of code without entering a function call.
- **step (s):** Executes the next line of code and enters the function if one is called.
- **print (p):** Displays the current value of a variable or expression.

- **display:** Automatically shows the value of a variable after each debugging step.
- **continue (c):** Continues the execution of the program until the next breakpoint or until the program finishes.
- **list (l):** Displays the source code around the current line being executed.
- **info locals:** Shows the values of all local variables in the current function.
- **backtrace (bt):** Displays the sequence of function calls that led to the current point or error.
- **info breakpoints:** Lists all the breakpoints that have been set in the program.
- **delete:** Removes one or more breakpoints.
- **disable:** Temporarily disables a breakpoint without deleting it.
- **enable:** Re-enables a previously disabled breakpoint.
- **quit (q):** Exits the GDB debugger.

Incorrect Code 4

```
#include <stdio.h>

int main()
{
    int i, sum = 0;

    for(i = 1; i < 5; i++)
    {
        sum = sum + i;
    }

    printf("Sum = %d\n", sum);

    return 0;
}
```

Screenshot of Wrong Code



```
(kali㉿kali)-[~]
$ gcc -Wall -Wextra SAMPLE4.C -o SAMPLE4

(kali㉿kali)-[~]
$ ./SAMPLE4
Sum = 10
```

Error Type

Wrong loop condition

Error Message / Output

Actual output: 10 instead of 15

Explanation

Condition should be $i \leq 5$ instead of $i < 5$.

Correct Code


```
#include <stdio.h>

int main()
{
    int i, sum = 0;

    for(i = 1; i <= 5; i++) // Correct condition
    {
        sum = sum + i;
    }

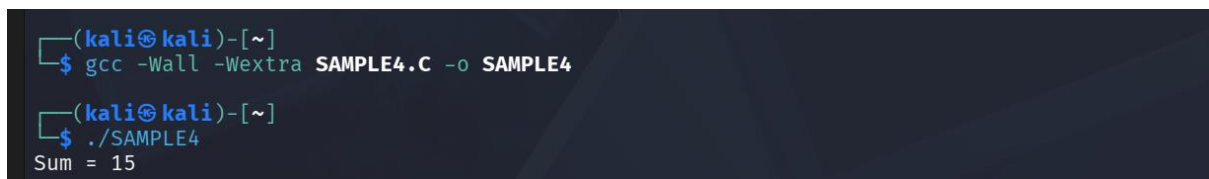
    printf("Sum = %d\n", sum);

    return 0;
}
```

Correct Output

Sum = 15

Screenshot of Correct Output



```
(kali㉿kali)-[~]
$ gcc -Wall -Wextra SAMPLE4.C -o SAMPLE4

(kali㉿kali)-[~]
$ ./SAMPLE4
Sum = 15
```

Conclusion

After correcting the code, the program works as expected.

Debugging Functions (next vs. step)

- next: Executes the line. If the line contains a function call, the function executes entirely in the background without entering its body.
- step: Steps into the function call, transferring GDB's focus into the function body for line-by-line inspection.

Incorrect code 5

```
#include <stdio.h>

int square(int x)
{
    int result1;

    result1 = x * 2; // Wrong calculation

    return result1;
}
```

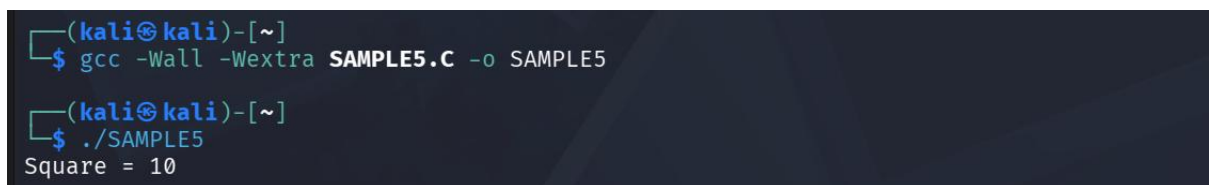
```

}

int main()
{
    int result, n = 5;
    result = square(n);
    printf("Square = %d\n", result);
    return 0;
}

```

Screenshot of Wrong Code



```

(kali㉿kali)-[~]
$ gcc -Wall -Wextra SAMPLE5.C -o SAMPLE5

(kali㉿kali)-[~]
$ ./SAMPLE5
Square = 10

```

Error Type

Debugger demonstration

Error Message / Output

No code error; demonstrates stepping.

Explanation

Use 'next' to step over and 'step' to step into functions.

Correct Code

```

#include <stdio.h>

int square(int x)
{
    int result1;
    result1 = x * x; // Correct calculation
    return result1;
}

int main()
{
    int result, n = 5;
    result = square(n);
    printf("Square = %d\n", result);
}

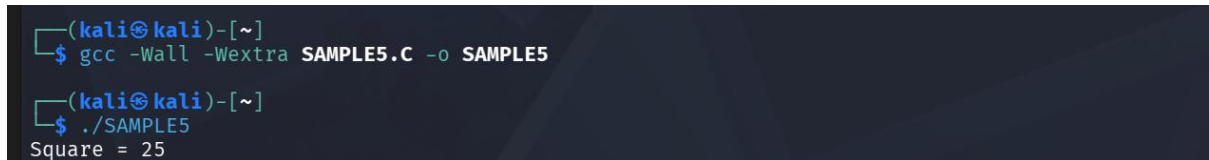
```

```
    return 0;
}
```

Correct Output

Square = 25

Screenshot of Correct Output



```
(kali㉿kali)-[~]
$ gcc -Wall -Wextra SAMPLE5.C -o SAMPLE5

(kali㉿kali)-[~]
$ ./SAMPLE5
Square = 25
```

Conclusion

After correcting the code, the program works as expected.

Understanding Stack Frames

- A stack frame contains information about a function that is currently being executed.
- When one function calls another, a new stack frame is created.
- Frame #0 represents the function where the error or crash actually occurred.
- Frame #1 represents the function that called Frame #0.
- Frame #2 represents the main function or the function that started the program.
- Stack frames help programmers trace the execution flow and identify the exact location of an error.

Incorrect Code 6

```
#include <stdio.h>

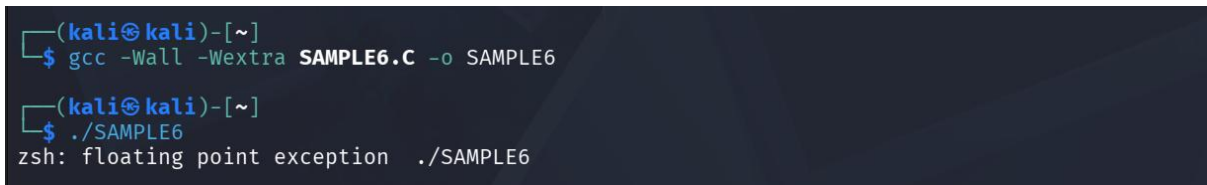
int divide(int a, int b)
{
    return a / b;    // Division by zero when b = 0
}

int calculate(int x)
{
    return divide(x, 0);
}

int main()
{
    int result = calculate(10);
    printf("Result: %d\n", result);
}
```

```
    return 0;
}
```

Screenshot of Wrong Code

A screenshot of a terminal window with a dark background. The prompt is (kali@kali)-[~]. The first command is \$ gcc -Wall -Wextra SAMPLE6.C -o SAMPLE6. The second command is \$./SAMPLE6. The output is zsh: floating point exception ./SAMPLE6.

```
(kali@kali)-[~]
$ gcc -Wall -Wextra SAMPLE6.C -o SAMPLE6

(kali@kali)-[~]
$ ./SAMPLE6
zsh: floating point exception ./SAMPLE6
```

Error Type

Division by zero

Error Message / Output

SIGFPE / Arithmetic exception

Explanation

Dividing by zero causes a runtime crash.

Correct Code

```
#include <stdio.h>

int divide(int a, int b)
{
    if (b == 0)
    {
        printf("Division by zero is not allowed.\n");
        return 0;
    }
    return a / b;
}

int calculate(int x)
{
    return divide(x, 2);
}

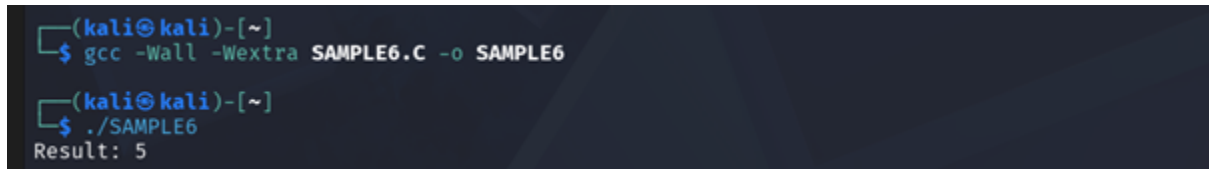
int main()
{
    int result = calculate(10);
    printf("Result: %d\n", result);
}
```

```
    return 0;
}
```

Correct Output

Result after valid divisor

Screenshot of Correct Output



```
(kali㉿kali)~[~]
$ gcc -Wall -Wextra SAMPLE6.C -o SAMPLE6
(kali㉿kali)~[~]
$ ./SAMPLE6
Result: 5
```

Conclusion

After correcting the code, the program works as expected.

7: Debugging Array Out-of-Bounds Errors using GDB

An array out-of-bounds error occurs when a program tries to access an array element outside its valid range. In C, the compiler does not automatically check array boundaries. Accessing an invalid index can cause undefined behaviour, memory corruption, or even a program crash. GDB helps programmers inspect array addresses and identify invalid memory access. To avoid this error, always use the correct loop condition and ensure that array indices remain within the valid range.

Incorrect code 7

```
#include <stdio.h>

int main()
{
    int a[5];

    int i;

    for(i = 0; i <= 5; i++)
    {
        scanf("%d", &a[i]);
    }

    return 0;
}
```

Screenshot of Wrong Code

```
(kali㉿kali)-[~]  
$ gcc -Wall -Wextra SAMPLE7.C -o SAMPLE7  
  
(kali㉿kali)-[~]  
$ ./SAMPLE7
```

Error Type

Array index error

Error Message

Array Out-of-Bounds Error

Explanation

Valid indices are 0 to 4. It will not give us output

Correct Code

```
#include <stdio.h>  
  
int main()  
{  
    int a[5];  
    int i;  
    for(i = 0; i < 5; i++)  
    {  
        scanf("%d", &a[i]);  
    }  
    return 0;  
}
```

Correct Output

Runs safely

Screenshot of Correct Output

```
(kali㉿kali)-[~]  
$ gcc -Wall -Wextra SAMPLE7.C -o SAMPLE7  
  
(kali㉿kali)-[~]  
$ ./SAMPLE7
```

Conclusion

After correcting the code, the program works as expected.

8: Segmentation Faults (SIGSEGV) using GDB

- A Segmentation Fault (SIGSEGV) occurs when a program tries to access memory that it is not allowed to access. This usually happens because of a NULL pointer, an uninitialized pointer, or invalid memory access. GDB helps locate the exact line where the segmentation fault occurs. By checking pointer values in GDB, programmers can identify the cause of the crash and correct the program. Proper pointer initialization and valid memory access help prevent segmentation faults.

Incorrect code 8

```
#include <stdio.h>

int main()
{
    int *p = NULL; // Pointer points to NULL

    *p = 10;       // Invalid memory access

    printf("%d\n", *p);

    return 0;
}
```

Screenshot of Wrong Code

A screenshot of a terminal window with a dark background. The prompt is '(kali@kali)-[~]'. The first command is '\$ gcc SAMPLE8.C -o SAMPLE8'. The second command is '\$./SAMPLE8'. The output is 'zsh: segmentation fault ./SAMPLE8'. There is a faint 'KALI' logo in the background.

Error Type

NULL pointer dereference

Error Message / Output

Segmentation fault

Explanation

Writing through a NULL pointer causes segmentation fault.

Correct Code

```
#include <stdio.h>

int main()
{
    int x;

    int *p = &x; // Pointer points to a valid variable
```

```
*p = 10;

printf("%d\n", *p);

return 0;
}
```

Correct Output

10

Screenshot of Correct Output



```
(kali@kali)-[~]
$ gcc SAMPLE8.C -o SAMPLE8
(kali@kali)-[~]
$ ./SAMPLE8
10
```

Conclusion

After correcting the code, the program works as expected.

Systematic Debugging Strategy

- Compare the expected output with the actual output.
- Reproduce the error to understand when it occurs.
- Compile the program using debugging options such as `-Wall`, `-Wextra`, and `-g`.
- Identify the part of the code where the error is likely to occur.
- Use `printf()` statements or GDB to inspect variables and program execution.
- Apply only the necessary changes to fix the error.
- Recompile and test the program again.
- Finally, verify the program with different test cases to ensure that the error has been completely removed.

EXERCISE AND HOMEWORK QUESTIONS

ERROR CLASSIFICATION :

Syntax Error (Missing semicolon):

```
#include <stdio.h>
```

```
int main() {
```

```
    int x = 10
```

```
    printf("%d\n", x);
```

```
    return 0;
```

```
}
```


Compilation / Type Error (Type mismatch):

```
#include <stdio.h>

int main() {
    int *ptr = "Hello World";
    return 0;
}
```

Linker Error (Missing function definition):

```
#include <stdio.h>

void missingFunction(); // Declared but not defined

int main() {
    missingFunction();
    return 0;
}
```

Runtime Error (Division by zero):

```
#include <stdio.h>

int main() {
    int a = 10, b = 0;
    int result = a / b;
    return 0;
}
```

Logical Error (Incorrect loop condition):

```
#include <stdio.h>

int main() {
    // Intended to loop 5 times, but loops 4 times
    for (int i = 0; i < 4; i++) {
        printf("Iteration %d\n", i);
    }
    return 0;
}
```

2. GNU Concept

GNU Project: Initiated by Richard Stallman in 1983, the GNU Project aims to give computer users freedom and control by developing a completely free, open-source Unix-like operating system.

Role of GDB: GDB (GNU Debugger) is the standard source-level debugger for the GNU software ecosystem. It allows developers to inspect program execution, view variables, and analyze memory during runtime or post-crash analysis.

3. Shell Context

Kernel: The central core component of an operating system that manages hardware resources (CPU, memory, storage) and mediates communication between software applications and system hardware.

Shell: A user-interface program (command-line or graphical) that acts as an interpreter, taking input commands from the user and translating them into instructions for the kernel to execute.

Three Common Linux Shells:

Bash (Bourne Again Shell)

Zsh (Z Shell)

Fish (Friendly Interactive Shell)

4. Differences: Breakpoint, Watchpoint, Catchpoint, and Tracepoint

Breakpoint: Pauses execution when the program reaches a specific source line, function, or memory address.

Watchpoint: Pauses execution whenever the value of a specified variable or memory location changes (or is read).

Catchpoint: Pauses execution when a specific C++ exception or system event occurs (e.g., throwing an exception, catching a signal, or loading a dynamic library).

Tracepoint: Logs specific program data at specified points without pausing or stopping program execution (commonly used in remote or performance-critical debugging).